

Chapter 5. Policy Gradient Methods

[Chapter 2](#) introduced value methods, which allow agents to learn the expected return by visiting each state-action pair. The agent can then choose the most valuable action at each time step to maximize the reward. You saw three different ways of estimating the expected return—Monte Carlo, dynamic programming, and temporal-difference methods—but all attempt quantify the value of each state.

Think about the problem again. Why do you want to learn expected values? They allow you to iterate toward an optimal policy. Q-learning finds the optimal policy by repeatedly maximizing the next expected value. But this is an indirect route to a policy. Is it possible to find the optimal policy directly? The answer is yes. Policy-based methods allow the agent to select actions without consulting a value function and learn an optimal policy directly.

Benefits of Learning a Policy Directly

[“Optimal Policies”](#) defined the policy as the probability of an action given a state. Q-learning delivered that policy by driving actions to produce the optimal policy. This simplification is appropriate for deterministic problems. Take the card game blackjack (also called 21 or pontoon), for example. You play by deciding whether to draw another card to increase the combined sum of the numbers on the cards, or not, by *sticking* with what you have. With a score of 22 or higher you lose the game by going bust. Another player does the same and the highest score wins.

One deterministic policy is to stick when you reach 21, otherwise you will lose every time. Value methods are appropriate for this problem. But what about when your reward is 17 and the other player is looking nervous? In a real game you would make a prediction, based upon the observation of the environment, which is often stochastic in real-life problems. A deterministic policy would choose the same action every time, irrespec-

tive of the chance that the person might be bluffing. But a stochastic policy could choose randomly between the two options, using the stochastic observation as evidence toward one or the other.

Using a stochastic policy provides another benefit; it neatly encapsulates the exploration dilemma. The agent can select actions according to arbitrary probabilities, removing the need for forced exploration strategies like ϵ -greedy. And for problems that include function approximation (any algorithm predicting Q-values), the best approximate policy could be stochastic.

A further benefit is that stochastic policy action preferences change smoothly over time. Action-value methods could chatter between two almost equally likely actions, creating unstable transitions. This occurs even when action-value estimates are relatively stable; minor changes in the Q-values are enough to dramatically change the policy. In contrast, stochastic policies provide the capability of probabilistic actions, which may result in more robust policies.

But the key difference is that the policy might be easier to approximate. Industrial problems tend to span a spectrum. Sometimes approximating the action-value function is easier, other times directly obtaining the policy is. Where the policy is easier to approximate, policy-based methods will learn faster.

How to Calculate the Gradient of a Policy

The goal of this section is to establish a method to derive an optimal policy. In general, you cannot use an analytic method; policies are potentially nonlinear and interactions with the environment make the problem non-stationary (which means that the distribution of the data changes).

[Chapter 4](#) introduced the idea of using a function to approximate the action-value estimates. This was possible because the agent could store previous interactions with the environment for later use. You can leverage a similar procedure to find an optimal policy, but first you need to define what you are trying to solve.

Recall that the goal of reinforcement learning (RL) is to create an optimal policy, π^* , which maximizes the expected return, G . If you had full observability of all possible trajectories, you could derive the optimal policy directly. But in general the number of permutations is too high (or infinite for continuous states or actions). Instead, you can create a parameterized model of the policy and tinker with the parameters until you find a good policy.

For example, a reasonable long-term index buying strategy could be to hold when the market is increasing and buy when the market is dropping. This way you gain when the market gains and you buy cheap assets when the market drops. Assuming that the index will continue to increase in the long term, this is a safe, low-volatility strategy. The model for such a policy could be a linear coefficient describing the current trend of the index. You could find the optimal policy by testing lots of different values for the linear parameter. But how should you quantify performance?

In the field of optimization, performance is a function, $J(\theta)$, parameterized by a model's parameters, θ . [Equation 5-1](#) is the objective function in this case, which is the expectation of the total reward when following a policy, π .

Equation 5-1. The policy objective function

$$J(\theta) \doteq \mathbb{E}_{\pi} G \mid s, a]$$

Assuming for a moment that you can calculate the objective function, then the challenge is to nudge the parameters of the policy to maximize the return. One popular method to achieve this is to use the gradient of the objective function, with respect to the parameters, $\nabla_{\theta} J(\theta)$, to direct improvements. This is called *vanilla gradient ascent* and is shown in [Equation 5-2](#) (note that gradient descent is the same as gradient ascent but with the sign changed to move downhill, instead of uphill).

Equation 5-2. Vanilla gradient ascent

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Policy Gradient Theorem

To restate the previous section, you are trying to calculate the gradient of the total reward, with respect to the policy parameters. This would tell you how to nudge the parameters to increase the reward.

But this depends on calculating the gradient of the objective function ([Equation 5-1](#)). This isn't as straightforward as it might seem, because the derivative requires some slightly complicated mathematics. But the main issue is that gradient ascent assumes the distribution of the data is stationary. A policy-based agent is going to be updating the policy at the same time as generating trajectories. This is like trying to pull a rug while standing on it. I will tackle the nonstationary problem later; for now let me carry on with the gradient derivation.

I think it is important that you see how the gradient is derived so I have worked hard to simplify the derivation as much as possible. It is best to read this derivation line by line, referring to the explanation underneath. You can find a formal derivation in the resources found in [“Further Reading”](#). I have removed the references to θ ; in all cases the policy, π_θ , is defined by its parameters, θ . Also note that this derivation is for the undiscounted case. The result of this derivation is shown in [Equation 5-3](#).

Equation 5-3. The gradient of a policy

$$\nabla J(\theta) \doteq \nabla \mathbb{E}_\pi [G \mid s, a] \quad (1)$$

$$= \nabla \sum_{a \in \mathcal{A}} Q(s, a) \pi(a \mid s) \quad (2)$$

$$\propto \sum_{a \in \mathcal{A}} Q(s, a) \nabla \pi(a \mid s) \quad (3)$$

$$\propto \sum_{a \in \mathcal{A}} Q(s, a) \pi(a \mid s) \frac{\nabla \pi(a \mid s)}{\pi(a \mid s)} \quad (4)$$

$$\propto \sum_{a \in \mathcal{A}} Q(s, a) \pi(a \mid s) \nabla \ln \pi(a \mid s) \quad (5)$$

$$\propto \mathbb{E}_\pi [G \nabla \ln \pi(a \mid s)] \quad (6)$$

The goal is to find the gradient of the objective function ([Equation 5-1](#)), which I describe in step (1).

In step (2), I have replaced the expected return with an implementation. You could pick any, but it makes the math a bit easier if you choose the the action-value function. The expected return is equal to the value of

each action, Q , multiplied by the probability of visiting that action (as defined by the policy, π).

WARNING

[Equation 5-1](#) has a large amount of mathematics between step (2) and (3). It results in a formality that the gradient is dependent on the distribution of states. In other words, the gradient is proportional to the probability of being in a certain state; if it is impossible to land in a state then it doesn't affect the gradient. Therefore, the gradient is only proportional to changes in the policy. I decided to hide these steps and the state distribution because eventually it cancels out due to the return of the expectation. If you are interested in the formal proof, please refer to the resources found in ["Further Reading"](#).

The gradient is taken with respect to the policy parameters, so I can move the action-value function, Q , outside of the gradient calculation in step (3).

Step (4) uses a cheeky mathematical trick that is only obvious with hindsight. I have multiplied the whole equation by $\pi(a | s)/\pi(a | s)$, which is equal to one, so it doesn't alter the equation.

Step (5) uses a common mathematical identity, $\nabla \ln x = \nabla x/x$, to replace the fraction with a natural logarithm. This is a common computational trick to prevent the multiplication of small probabilities driving the gradients to zero (this is called the vanishing gradients problem).

Finally in step (6), recall that the sum of the action values multiplied by the action probabilities is precisely what I started with in step (2). So rather than summing over all actions, I can instead tell the agent to follow the policy and return the expectation of that particular action over time.

There you are: the derivation of the policy gradient algorithm. To summarize the result, the gradient of the return, with respect to the policy, is equal to the expected value of the return, multiplied by the gradient of the policy. In other words, the expected return magnifies the gradient of the policy. This makes sense because we prefer to nudge the parameters of the policy more when the expected value is higher.

The key insight is that the return, G , can be estimated in a wide variety of ways. This is the the only difference between many famous policy gradient algorithms. I return to this in [“A Comparison of Basic Policy Gradient Algorithms”](#).

Step (6) in [Equation 5-3](#) is a quantity that can be sampled on each time step, whose expectation is equal to the gradient. You can plug this into the gradient ascent algorithm ([Equation 5-2](#)) to produce an update rule for an agent, like in [Equation 5-4](#).

Equation 5-4. Policy gradient update rule

$$\theta \leftarrow \theta + \alpha G \nabla \ln \pi(a \mid s, \theta)$$

Policy Functions

Refer back to [Equation 5-4](#). Two key details are missing from the policy gradient algorithm. The first is the definition of G , which I discuss in [“Basic Implementations”](#).

The second is the gradient of a policy. I need to calculate $\nabla \ln \pi(a \mid s, \theta)$ and to do this I need to specify my policy function.

I must admit that I despair at the need to design another model, but the choice is important. The policy defines how well your agent can plan through all the possible trajectories in your environment. It needs to be complex enough to handle complex trajectories, but simple enough to learn. There has been some focus on attempting to automate this process.¹

But the major benefit, and one of the major reasons why policy gradient methods are popular, is that you can choose *any* function, as long as it outputs a probability distribution and is differentiable with respect to θ . The world is your oyster, if models are your pearls.

Linear Policies

If you are using a library that performs automatic differentiation, like TensorFlow or PyTorch via back-propagation, then you don't even need to think about the derivative. But I find it helps to think about a simple case to provide the intuition.

Logistic policy

You need a function that predicts an action given a state: $\pi_{\theta}(a | s)$. Recall that the policy has two requirements: it must output probabilities and it must be differentiable. Imagine for a second that you are tackling a problem that has two discrete actions, like the `CartPole` environment, where you have to decide whether to move the cart to the left or right (see [“Example: DQN on the CartPole Environment”](#) for a description of the environment). The problem becomes a task of predicting one of two classes, given some inputs; it must output probabilities. The natural choice for this type of problem is the logistic function, commonly used in classification, because it is simple, linear, easy to differentiate, and naturally outputs probability estimates (according to the logistic distribution, which has larger tails when compared to the normal distribution). The logistic function, centered around zero with a maximum value of 1, is shown in [Equation 5-5](#) (also known as the *sigmoid function*).

Equation 5-5. Logistic function

$$f(x) \doteq \frac{1}{1 + e^{-x}}$$

The `CartPole` environment has four states, so θ is a vector of four weights. To use this policy function, feed in the state and the current estimates for the weights and it will output the probability of selecting an action. In the beginning, the weights are randomly distributed and during training they are nudged toward selecting better actions. These steps will sound familiar if you are used to doing supervised machine learning.

In general you can use multiple logistic functions to model the probability in multi-class problems (this is equivalent to the softmax function that I use in [“Softmax policy”](#)). But since this environment has only two actions, and the output of [Equation 5-5](#) spans from zero to one, you can assign the probability of one class equal to $f(\theta, s)$ and the probability of the other

equal to $1 - f(\theta, s)$. This is *dummy encoding*, which is used in data cleaning. A policy that predicts an action, a , given states, s , based upon the logistic function, is shown in [Equation 5-6](#).

Equation 5-6. Logistic policy

$$\pi_{\theta}(a | s) \doteq \frac{\pi_{\theta}(a = 0 | s)}{\pi_{\theta}(a = 1 | s)} = \frac{\frac{1}{1+e^{-\theta_0^T s}}}{1 - \frac{1}{1+e^{-\theta_1^T s}}}$$

[Equation 5-4](#) provides the equation to improve the weights, but you need the gradient of [Equation 5-6](#) to do that. The result of this is shown in [Equation 5-7](#), but you can see the full calculation in [Appendix A](#).

Equation 5-7. Logistic policy gradient

$$\nabla \ln \pi(a | s, \theta) = \frac{s - s\pi(\theta_0^T s)}{-s\pi(\theta_1^T s)}$$

The gradient has an intuitive explanation. [Equation 5-7](#) is the difference between the observed state and the state expected by the policy.

Softmax policy

You can think of the softmax function as a multiclass version of the logistic function. This means you can use this to represent the probability of choosing from multiple actions. A softmax policy is defined in [Equation 5-8](#).

Equation 5-8. Softmax policy

$$\pi_{\theta}(a | s) \doteq \frac{e^{\theta_a^T s}}{\sum_a e^{\theta_a^T s}}$$

In other words, [Equation 5-8](#) is a logistic-like function normalized over all possible actions to ensure that all actions sum to one. The corresponding gradient is shown in [Equation 5-9](#) and you can see the full derivation in [Appendix B](#).

Equation 5-9. Softmax policy gradient

$$\nabla \ln \pi(a | s, \theta) = s - \sum_a s\pi(\theta_a^T s)$$

Like [“Logistic policy”](#), the gradient shown in [Equation 5-9](#) works out as the difference between the observed and expected feature vectors.

Arbitrary Policies

Computing the derivatives of policies by hand has allowed me to show you precisely how I can improve a policy via iteration. But in general, this process is both tedious and obfuscating. Instead, you could compute a derivative in the following three ways: finite, automatic, and symbolic.

Finite differences calculate the difference in a policy over a small step in the parameter space, $\nabla f(\mathbf{x}) = (f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})) / 2\mathbf{h}$. This is simple but prone to numerical error because of the step size, $\mathbf{h}$.

Automatic differentiation tools, like [JAX](#) or the back-propagation methods in TensorFlow/PyTorch/MXNet/others, attempt to track the usage of a parameter in an equation to build a directed acyclic graph (DAG). The DAG libraries map each step to the appropriate derivative and combine them through the chain rule.

Symbolic differentiation, using tools like [SymPy](#), is effectively the same as automatic differentiation except it generally works in one direction (forward). Symbolic differentiation tools also place more emphasis on refactoring, which can result in derivatives that are easier to understand, as opposed to automatic methods that can result in chains of chains of chains, and so on.

In other words, if you can leverage a library, I recommend that you don’t do the differentiation by hand. Use a library that suits your use case.

Basic Implementations

[“Policy Functions”](#) showed you how to provide the gradient function required by [Equation 5-4](#). The remaining detail is the use of G . Recall that this is the total reward from a point up until the end of an episode, if episodic, or forever more, if not (see [“Discounted Rewards”](#)).

Chapters [2–4](#) show three different ways of calculating expected return, from Monte Carlo, full trajectory methods through to approximation,

multistep projection methods. These implementation details are the only difference between several basic policy gradient implementations, including the REINFORCE and actor-critic algorithms.

Monte Carlo (REINFORCE)

The first algorithm uses a Monte Carlo approach. It first collects a trajectory, based upon the current policy, for an entire episode. It then updates the policy via [Equation 5-4](#).

[Algorithm 5-1](#) shows the REINFORCE algorithm. In step (6), G refers to the discounted reward from a time t up until the end of the episode, like in [Algorithm 2-3](#). In step (7), I use the value of G to weight the differential of the policy. The differential calculates the direction in which you need to nudge the probability of choosing an action, defined by θ . The collective weighting of $\alpha\gamma^t G$ defines how much you need to nudge it by. I also reintroduce discounting in step (7), which is pulled out of the derivative because it does not depend on θ .

The issue here is that REINFORCE will continue to promote an action in proportion to the calculated return. Even when an agent made the right choice, REINFORCE will continue to push the probability of choosing that action even higher. In the long term, it can overweight the relative importance of an action compared to other actions and this affects learning performance. For example, all actions might have an equal expected return, but REINFORCE will always increase the probability of the last action.

Algorithm 5-1. REINFORCE algorithm

- 1: **input:** a differentiable parameterized policy $\pi(a | s, \theta)$, step size $\alpha > 0$
- 2: Initialize θ arbitrarily
- 3: **loop** for each episode:
- 4: Generate trajectory following $\pi(\cdot | \cdot, \theta)$
- 5: **for** each step in episode $t \leftarrow 0, 1, \dots, T$:
- 6: $G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} r_k$
- 7: $\theta \leftarrow \theta + \alpha \gamma^t G \nabla \ln \pi(a | s, \theta)$
- 8: **endfor**

Example: REINFORCE on the CartPole environment

The beauty of using policy gradients is demonstrated by the low dimensionality of the policy. The `CartPole` environment has four features. The linear policy has four parameters that weight the importance of each of the features. The result is a probabilistic representation of the best course of action. If the angle of the pole is close to falling over, then the probability of choosing an action to attempt to correct that is very high. The rewards obtained during the training process are shown in [Figure 5-1](#). The policy should look far into the future to obtain good rewards, so I set $\gamma \doteq 0.99$. I chose $\alpha \doteq 0.01$ because at higher values the learning was unstable.

Compare this result to the tabular implementation in [“Example: DQN on the CartPole Environment”](#). There, I had to use a neural network to approximate the state space. Here, I have a linear mapping using just four parameters. The elegance, the sheer utility, is incredible.

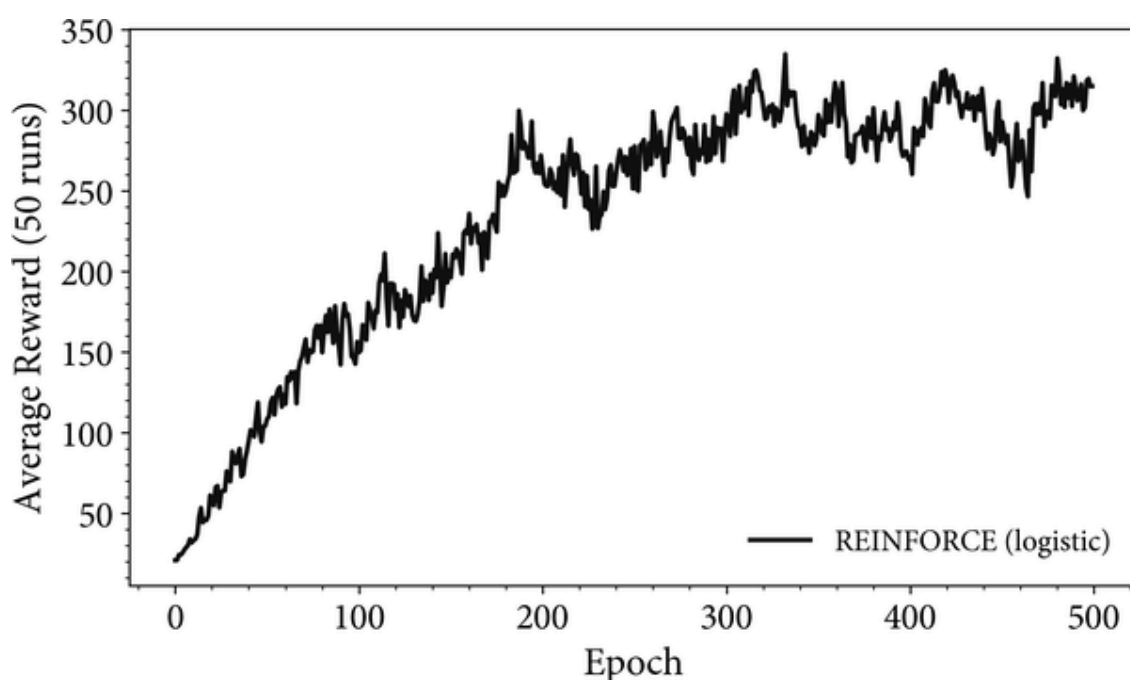


Figure 5-1. The average rewards using a REINFORCE-based policy gradient algorithm on the `CartPole` environment.

REINFORCE with Baseline

A more common implementation incorporates a baseline expected return, like the advantage function seen in [“Dueling Networks”](#) (see [Equation 4-4](#)). This helps to improve the instability of the gradients due to large changes in the average reward.

Imagine a scenario (like `CartPole`) that rewards on each time step. This creates high expected returns during early time steps in a trajectory. Large changes in value create large updates to the gradients (as defined by [Equation 5-4](#)). But the agent should not concern itself with the machinations of the average return; the goal is to find optimal actions. The agent should instead learn how each action improves the expected return, relative to the average.

Advantage algorithms implement this by maintaining a prediction of the current average return for a given state, which is the same as the state-value function (see [Equation 2-8](#)). And as before, you can approximate the state-value function in many different ways: tabular methods or function approximation, for example. Since this is a parameterized Monte Carlo algorithm, it is natural to update a parameterized state-value estimate in a similar way, by calculating the error between predicted and actual values and nudging the parameters according to the update rule, along full episode trajectories.

The net result is an improvement in learning stability. The subtraction of the baseline means each gradient update is less extreme, which leads to lower variance in the gradients. In the long term you should observe that the rewards obtained during learning are more stable. But this depends heavily on the complexity of the state space, whether it is possible to predict the state-value function and the complexity of the actions. In the `CartPole` example, both are very simple, so you should expect little difference over the standard REINFORCE algorithm.

[Algorithm 5-2](#) describes the REINFORCE algorithm with the baseline. The major difference is the addition of steps (7) and (8). Here, I train a second linear function to approximate the state-value function. Step (7) subtracts the approximation from the reward observed in this trajectory. When applied to the policy weights in step (9), this has the effect of altering the probability of choosing the action according to the difference. If the difference is greater than zero (the reward is higher than the baseline), the probability is increased. If the difference is less than zero, the probability is decreased.

Algorithm 5-2. REINFORCE with baseline algorithm

```
1: input: a differentiable parameterized policy  $\pi(a \mid s, \theta)$ ,  
    a differentiable parameterized state-value function  $V(s, w)$ ,  
    step sizes  $0 < \alpha_w < 1$  and  $0 < \alpha_\theta < 1$   
2: Initialize  $\theta$  and  $w$  arbitrarily  
3: loop for each episode:  
4:   Generate trajectory following  $\pi(\cdot \mid \cdot, \theta)$ :  
5:   for each step in episode  $t \leftarrow 0, 1, \dots, T$ :  
6:      $G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} r_k$   
7:      $\delta \leftarrow G - V(s, w)$   
8:      $w \leftarrow w + \alpha_w \delta \nabla V(s, w)$   
9:      $\theta \leftarrow \theta + \alpha_\theta \gamma^t \delta \nabla \ln \pi(a \mid s, \theta)$   
10:  endfor
```

Example: REINFORCE with baseline on the CartPole environment

There were few changes to the experiment in [“Example: REINFORCE on the CartPole environment”](#) except for the implementation of [Algorithm 5-2](#). The main architectural decision is the implementation of the state-value estimate, $V(s, w)$. In my experiments I was struggling to find a simple function to fit the data. This was because the underlying state is surprisingly complex.

To keep the experiment as simple as possible, I decided to use a rolling average of the previously observed rewards; in other words, an online implementation of the average reward. Then I retrained the agent and saved the state and discounted rewards. Then I performed principal component analysis on the state to reduce the number of dimensions from four to one, to help simplify the feature space and enable plotting.

NOTE

You can read more about dimensionality reduction and principal component analysis in any book on machine learning; some recommendations are in [“Further Reading”](#).

[Figure 5-2](#) shows the plot of the principal component of the state against the discounted reward. The rewards are forming a triangular shape around the state value of zero. The predicted value is higher if you keep

the state values around zero. In other words, if the state values are non-zero then this will, in general, produce a lower total reward, because the pole is falling over. I also find it amusing that the plot looks a lot like the *Eye of Providence* from the back of a dollar bill.

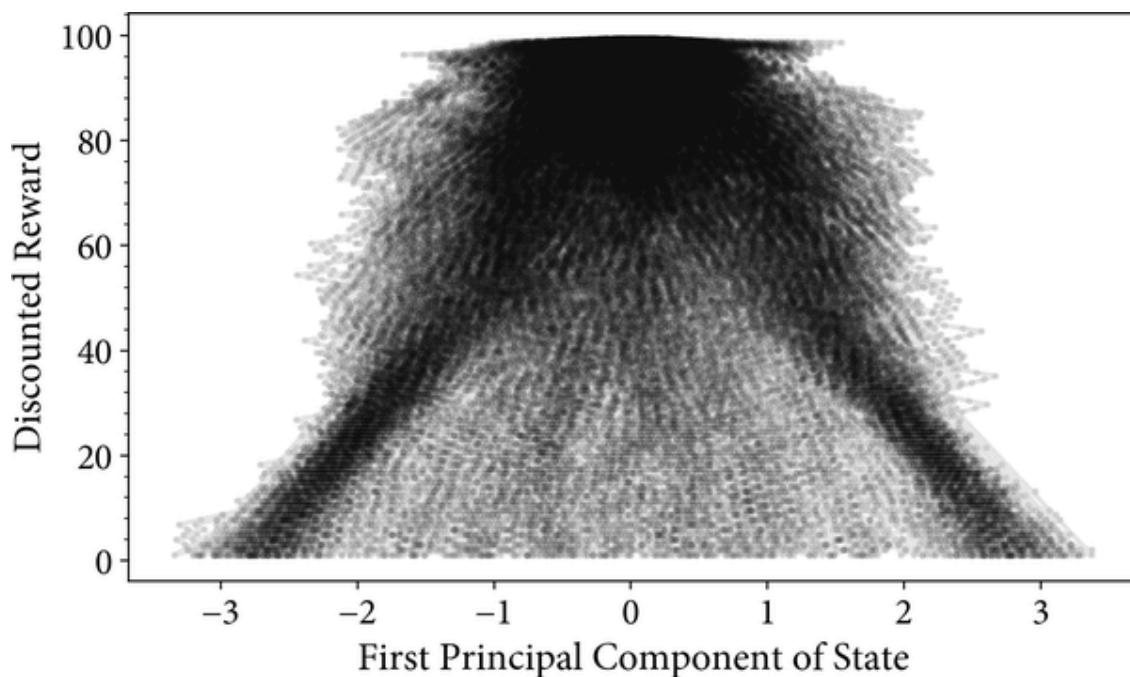


Figure 5-2. The first principal component of the `CartPole` state plotted against the discounted reward.

Irrespective of the coincidental iconography, you can see that a simple average or linear approximation will not predict the value well. Also, I have shown the first principal component for plotting purposes. The data probably contains complexity in the second, third, and fourth dimensions, too.

For now, I will stick with my choice of using the average and consider a more complicated approximation function in [“Example: n-step actor-critic on the CartPole environment”](#). But in general, it is important to consider this trade-off. You want to predict the state-value function as well as possible, but you also want the function approximation to be robust (in the sense that an unseen state should not produce a ridiculous result).

The results of my experiment can be seen in [Figure 5-3](#). I had to run the experiment over 50 different runs to obtain results that were statistically significant with the same settings. You can see that the reward initially tracks the standard REINFORCE algorithm, but later diverges. This indicates that the baseline algorithm is slightly more stable. In the REINFORCE algorithm, the large gradient updates accidentally knock the agent back toward worse performance. The baseline agent is able to steadily continue improving performance.

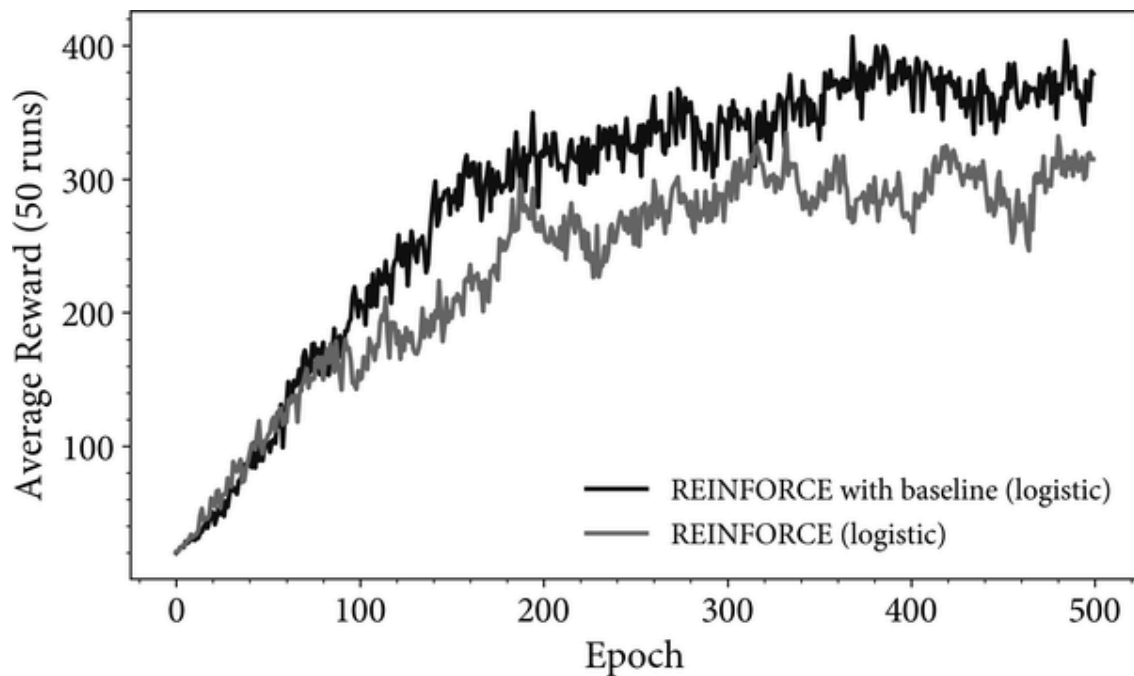


Figure 5-3. The average rewards using a REINFORCE with baseline policy gradient algorithm on the `CartPole` environment.

Gradient Variance Reduction

In [“REINFORCE with Baseline”](#) I stated that the baseline method decreases the variance of the gradients. In algorithms like REINFORCE or other basic Monte Carlo methods (see [“Monte Carlo Policy Generation”](#)), the goal is to alter the policy to maximize the reward. Policy gradient methods achieve this by nudging the policy in a certain direction and by a certain amount. They calculate the direction using the derivative of the policy. But the magnitude, for REINFORCE, is the current expected return.

The problem with this is that subsequent samples may experience large differences in the expected return. For example, imagine that a policy is reasonably efficient in the `CartPole` environment. The pole may, due to chance, accidentally fall over early in the episode. This will cause a large difference in the expected value and a large gradient update. Despite the fact that the policy was a good one, the agent forced a large update.

Another example is in the Pong environment from [“Example: Rainbow DQN on Atari Games”](#). Early episodes result in rewards close to -21. All actions result in a negative reward. Because all options are bad, the gradient update suppresses all actions. A better solution is to look at the change in the reward. The agent should use the relative difference of the actions compared to the average value.

I captured the gradient updates from a single run of the REINFORCE and REINFORCE with baseline algorithms. You can see the standard deviation of the gradients in [Figure 5-4](#). Notice that after a period of learning, the standard deviation of the baseline algorithm drops dramatically, whereas the REINFORCE algorithm continues to produce large gradients that alter the policy, even after obtaining decent rewards. When the literature talks about “variance reduction,” this is what they mean: methods to improve the stability of the gradients without affecting learning performance.

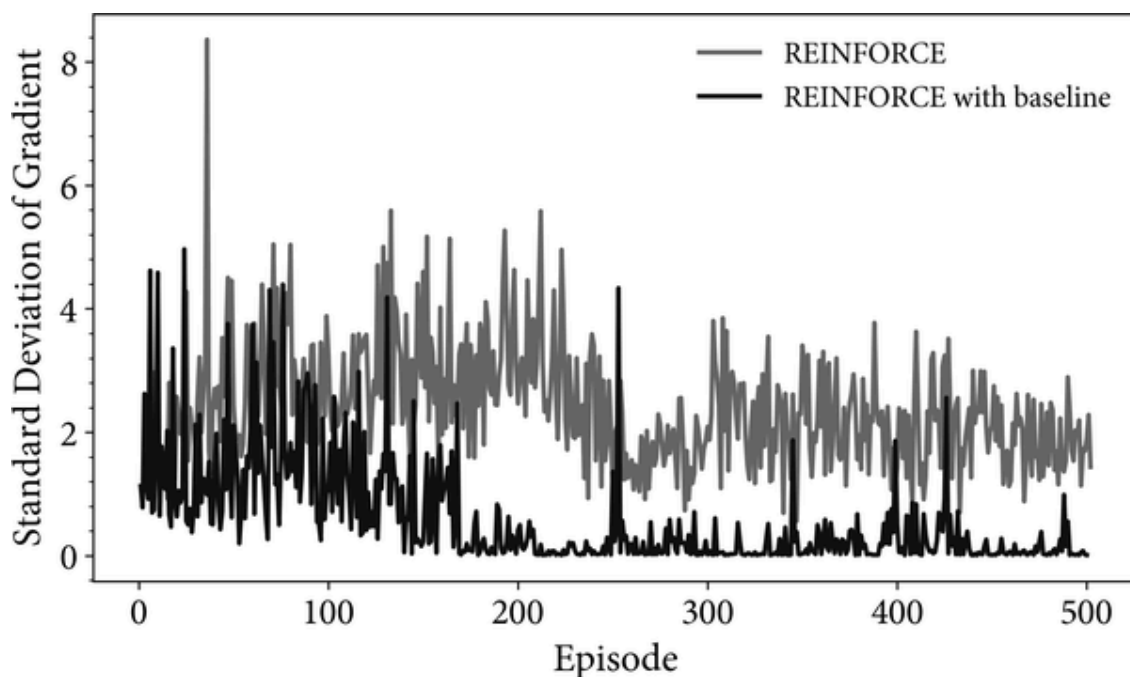


Figure 5-4. The average rewards using a REINFORCE with baseline policy gradient algorithm on the `CartPole` environment.

n-Step Actor-Critic and Advantage Actor-Critic (A2C)

The next improvement follows the path taken with tabular methods. Monte Carlo agents do not bootstrap, which means they have to wait until the episode is over before they can perform any learning. As in [Chapter 3](#), the solution is to work through temporal-difference methods up to n -step methods. However, policy gradient-based n -step algorithms have an interesting interpretation that you have not seen before.

Before I get there, let me describe the n -step implementation because there is one major change over [Algorithm 5-2](#). In step (5) of [Algorithm 5-3](#), the algorithm performs an update for every time step. This is no longer a Monte Carlo algorithm. Bootstrapping (see [“n-Step Algorithms”](#)) improves

learning speed and step (8) implements the bootstrapping, to provide one-step updates $G_{t:t+1} \doteq r + \gamma V(s', w)$, like in [Equation 3-3](#).

Algorithm 5-3. n -step actor-critic

```
1: input: a differentiable parameterized policy  $\pi(a | s, \theta)$ ,  
   a differentiable parameterized state-value function  $V(s, w)$ ,  
   step sizes  $\alpha_w$  and  $\alpha_\theta > 0$   
2: Initialize  $\theta$  and  $w$   
3: loop for each episode:  
4:   Initialize  $I \leftarrow 1, t \leftarrow 0$ , and  $s$   
5:   while  $s$  is not terminal:  
6:     Choose  $a$  from  $s$  using  $\pi$   
7:     Take action  $a$ , observe  $s', r$   
8:      $\delta \leftarrow G_{t:t+n} - V(s, w)$   
9:      $w \leftarrow w + \alpha_w \delta \nabla V(s, w)$   
10:     $\theta \leftarrow \theta + \alpha_\theta I \delta \nabla \ln \pi(a | s, \theta)$   
11:     $I \leftarrow \gamma I$   
12:     $s \leftarrow s'$   
13:     $t \leftarrow t + 1$ 
```

[Algorithm 5-3](#) is an incremental improvement over [Algorithm 5-2](#) that includes the temporal-difference return. But this algorithm is more than it seems. You have now derived an algorithm that can learn both a policy and its state-value function on every single step, as opposed to learning a baseline over entire trajectories (the Monte Carlo-inspired baseline algorithm).

This distinction is important. The policy learns and describes the best action to take. In RL parlance, this is known as the *actor*. The n -step baseline prediction criticizes the consequences of the action and is used to update the policy. This is the *critic*. Together they form the basis of *actor-critic* algorithms. Note that Monte Carlo algorithms are not considered to be actor-critic because they are not updated on every step.

This is more commonly called the advantage actor-critic (A2C) algorithm. I presented the algorithm in this way because you are familiar with development from [Chapter 3](#). But another way of formulating the problem is to consider the critic as a measure of the benefit of choosing an action, relative to a baseline. This is the exact definition of the advantage function from [Equation 4-4](#).

In the context of policy gradient algorithms, the only difference is that the agent follows the current optimal policy by definition, so you don't need to calculate the value of each action. You only need the value for the states that the policy is going to visit. An approximation (which is proven in the reference) is presented in [Equation 5-10](#).²

Equation 5-10. Advantage function approximation for policy gradient algorithms

$$\begin{aligned} A^\pi(s, a) &\doteq Q(s, a) - V(s) \\ &= r + Q(s', a') - V(s) && \text{(temporal-difference error)} \\ &\approx r + V_\pi(s') - V(s) && \text{(when the agent is following the optimal policy } \pi) \end{aligned}$$

It is possible to estimate the advantage function or the action-value function directly, but most implementations use the state-value version to reduce the number of models you need to train.

Another common question is the need for I in step (11) of [Algorithm 5-3](#). When including discounting (see [“Discounted Rewards”](#)) in the policy gradient theorem ([Equation 5-3](#)), γ is a constant on the outside of the gradient calculation. But the discounting depends on the step number and in an online algorithm like this it makes sense to implement it in this way, although you could simply replace it with a γ^t if you like.

Example: n-step actor-critic on the CartPole environment

The goal of this example is to demonstrate the differences between Monte Carlo (see [“REINFORCE with Baseline”](#)) and temporal-difference policy gradient implementations produced by the use of bootstrapping.

Recall that bootstrapping requires a function that estimates the expected return of a given state. If I used the rolling average of the reward, like I did in [“Example: REINFORCE with baseline on the CartPole environment”](#), then the estimates for $V(s, w)$ and $V(s', w)$ are the same. This yields $\delta = r + (1 - \gamma)V \approx r$. Using the previous baseline leads to updates that have a magnitude of 1 in the `CartPole` environment. In other words, the critic has no opinion. Instead, you must design a function that is more capable of predicting the value for a given state. The learning performance of an actor-critic algorithm is dependent on having an accurate value approximation.

You saw in [Figure 5-2](#) that the reward is a complex function of the state. A simple linear combination of the state and some weights is not capable of modeling this complexity. There is a trend to jump straight to a deep learning approach at this point, but I still recommend trying linear approaches first. They are more robust, faster to train, and harder to overfit. In the `CartPole` environment I can perform feature engineering on the state to obtain features that are learnable via a linear combination of weights.

After some experimentation I chose to use tile coding to convert the last two states (pole angle and angular velocity) into a grid of 10 cells per state with 5 overlaps (you can learn more about common feature engineering techniques in [“Converting to discrete states”](#)). This results in 605 features. I used 605 weights to learn a linear approximation of these features. The settings are the same as before except for the new decay parameter for the weights, $\alpha_w \doteq 0.1$. It is generally recommended that the value estimate should have a learning rate greater than or equal to the learning rate of the policy. In other words, the value estimate should learn at least as fast as the policy.

[Figure 5-5](#) shows the results of intentionally setting the policy learning rate, α_θ , to match the final learning performance of the previous algorithms.

[Figure 5-6](#) shows the results after increasing the policy decay rates (for both actor-critic and baseline algorithms) to $\alpha_\theta \doteq 0.1$. All other settings remain the same. The increase in decay rate means that the updates made to the policy are larger. The policy changes quicker. After removing the training wheels, you can see that the n -step actor-critic algorithm can keep up. It rapidly increases its reward over the first 50 episodes and settles around a reward of 400 (which is dependent on the modeling accuracy of the policy and the state-value function, not the learning rate). The baseline algorithm, however, is unable to learn any faster. The Monte Carlo updates significantly hamper learning performance. They act like a brake; changing the policy learning rate has no effect (and can affect long-term stability).

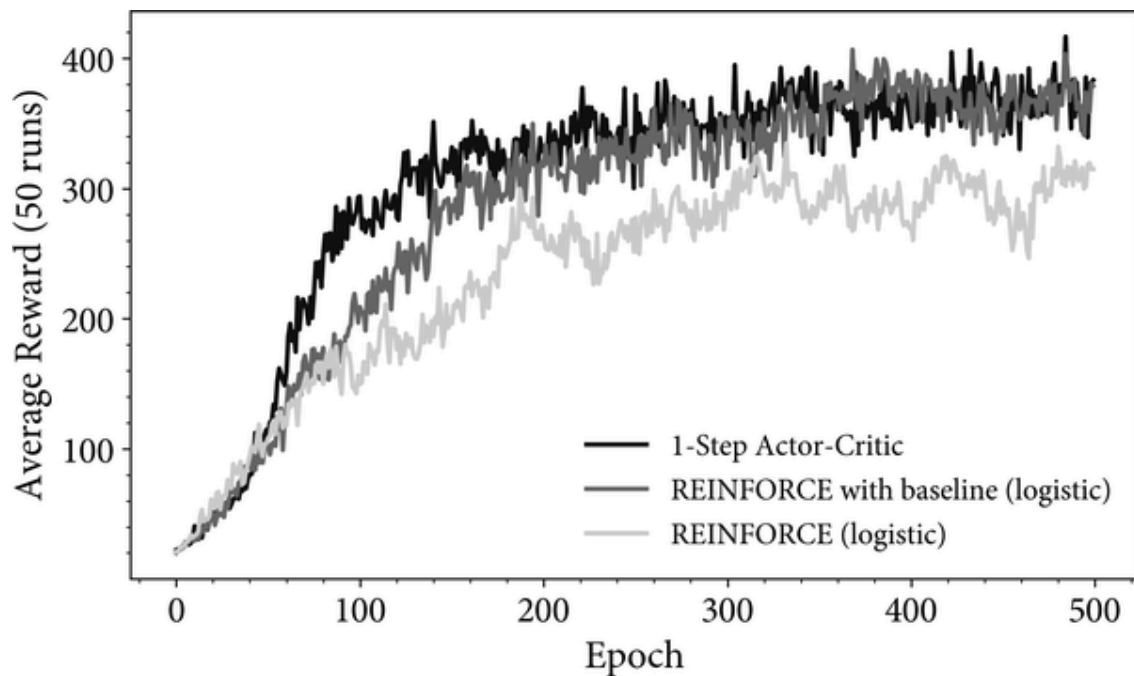


Figure 5-5. The average rewards using a 1-step actor-critic policy gradient algorithm on the `CartPole` environment.

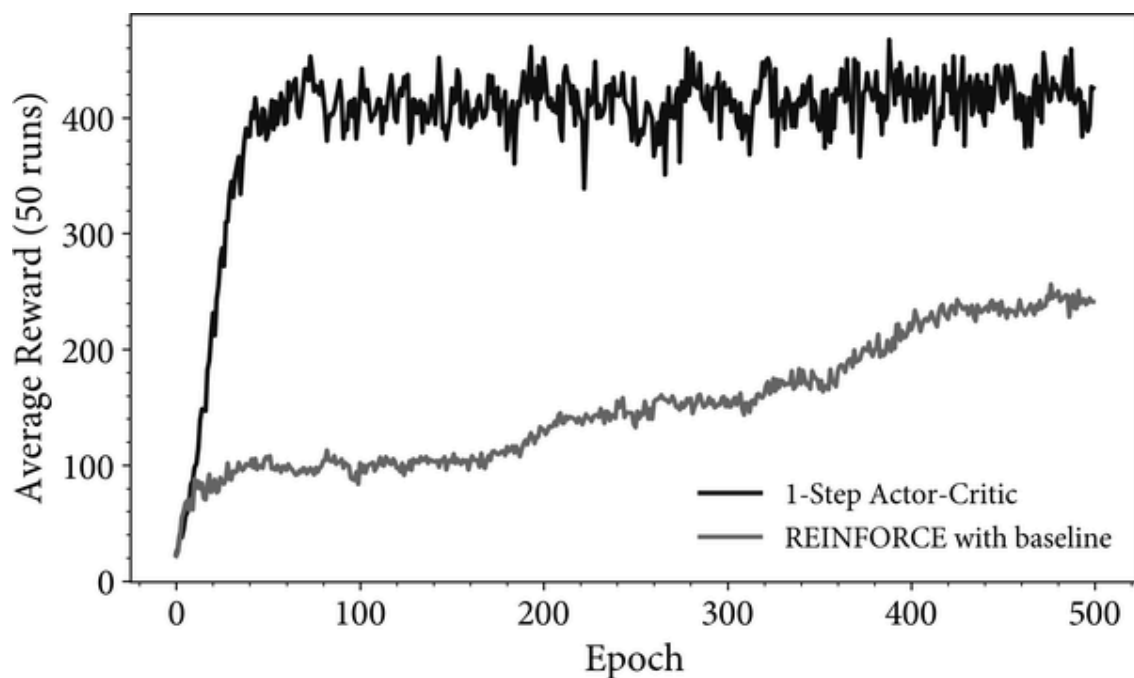


Figure 5-6. The average rewards for the `CartPole` environment after altering the policy decay rate (α_θ) from 0.01–0.1 for both actor-critic and baseline algorithms.

State-value learning decay rates versus policy decay rates

The decay-rate parameters denoted by α regulate the size of the jump in the update. Since this is a gradient descent algorithm, choose the decay rate to be large enough to learn quickly, but small enough to converge toward the best possible policy. But the n -step policy gradient algorithm uses two decay rates: one for the model that estimates the state value function and another for the policy.

The policy uses the state-value estimate in its update, which introduces a dependency. Setting the state-value estimate α to update slowly will constrain the policy. This is because the temporal-difference error in successive steps will be close to zero and result in a small change to the policy. Setting α to update quickly will cause excessive changes in the temporal-difference error and large changes in the policy. After the policy has changed, the true state value will also change and the state-value estimate will require another large update. The end result is instability and then the agent is unlikely to converge.

The results shown in [Figure 5-7](#) come from an experiment on the `CartPole` environment where I fixed the policy decay rate, $\alpha_\theta \doteq 0.1$, and altered the state-value estimate decay rate, $\alpha_w \doteq \{0.01, 0.1, 0.5\}$. The results show that the optimal decay rate is approximately 0.1, although a value of 0.01 can eke out greater performance in the long run because the smaller step size gets closer to the global optima. A value of 0.5 is too great; it causes unstable updates and prevents convergence.

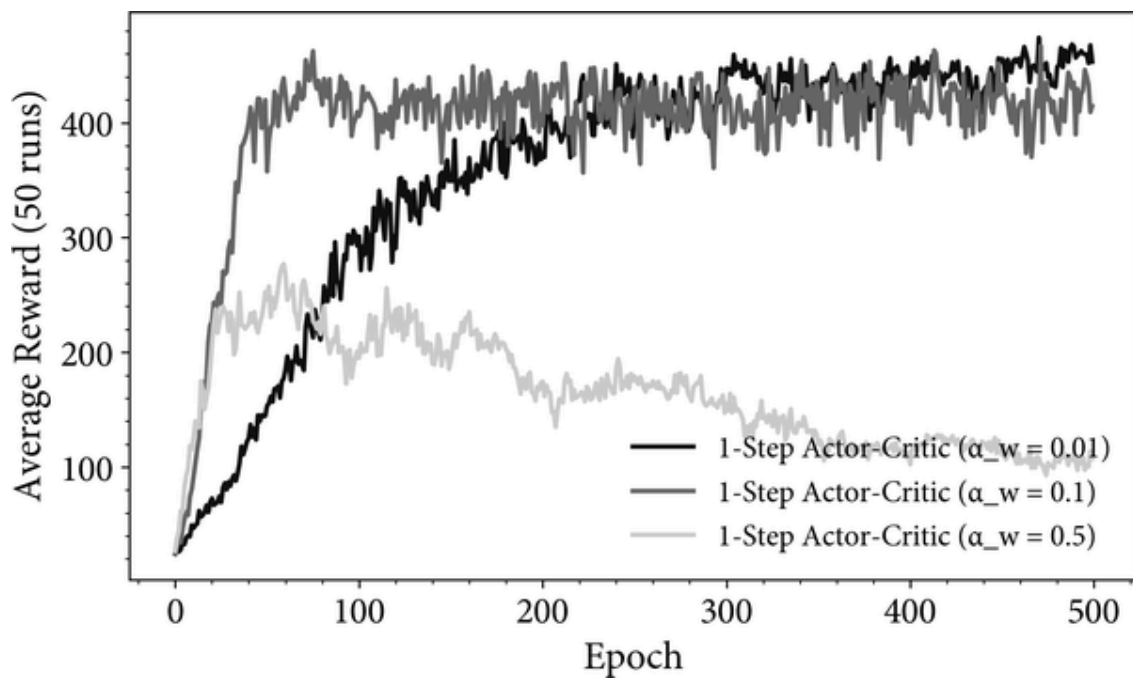


Figure 5-7. The average rewards for the `CartPole` environment when fixing the policy decay rate and altering the state-value decay rate.

There is a popular mathematical theorem called “no free lunch,” which I excessively summarize as proof that no single algorithm, implementation, or hyperparameters are best for all applications. Similarly, optimal decay rates are entirely dependent on the problem and the algorithm. You must design and tune agents for every new situation.³

Eligibility Traces Actor-Critic

Following the developments in [“Eligibility Traces”](#), [Algorithm 5-4](#) presents an actor-critic algorithm based upon eligibility traces. Like before, it exposes a hyperparameter to allow you to control the level of bootstrapping. The optimal value for the trace-decay hyperparameter is problem dependent. Large values update weights experienced far in the past. Small values only update recent weights. The major difference between [Algorithm 5-4](#) and [Algorithm 5-3](#) is the inclusion of the tracers that track which states to update in steps 9 and 10. These ultimately nudge the value estimate weights, w , and the policy weights, θ , in the direction that the policy visited in the past.

Algorithm 5-4. Actor-critic with eligibility traces

- 1: **input:** a differentiable parameterized policy $\pi(a | s, \theta)$,
a differentiable parameterized state-value function $V(s, w)$,
step sizes $0 < \alpha_w, \alpha_\theta < 1$, and trace-decay rates $0 \leq \lambda_\theta, \lambda_w \leq 1$
- 2: Initialize θ and w arbitrarily
- 3: **loop** for each episode:
- 4: Initialize $t \leftarrow 0, I \leftarrow 1$ and s ,
 $z_w \leftarrow 0$ (same dimensionality as w), and
 $z_\theta \leftarrow 0$ (same dimensionality as θ)
- 5: **while** s is not terminal:
- 6: Choose a from s using π
- 7: Take action a , observe s', r
- 8: $\delta \leftarrow r + \gamma V(s', w) - V(s, w)$
- 9: $z_w \leftarrow \gamma \lambda_w z_w + \nabla V(s, w)$
- 10: $z_\theta \leftarrow \gamma \lambda_\theta z_\theta + I \nabla \ln \pi(a | s, \theta)$
- 11: $w \leftarrow w + \alpha_w \delta z_w$
- 12: $\theta \leftarrow \theta + \alpha_\theta I \delta z_\theta$
- 13: $I \leftarrow \gamma I$
- 14: $s \leftarrow s'$
- 15: $t \leftarrow t + 1$

Example: Eligibility trace actor-critic on the CartPole environment

I made minor modifications to the example in [“Example: n-step actor-critic on the CartPole environment”](#) to include the decaying eligibility traces. This experiment uses $\alpha_\theta = \alpha_w = \lambda_\theta = \lambda_w \doteq 0.1$; you can observe the results in [Figure 5-8](#).

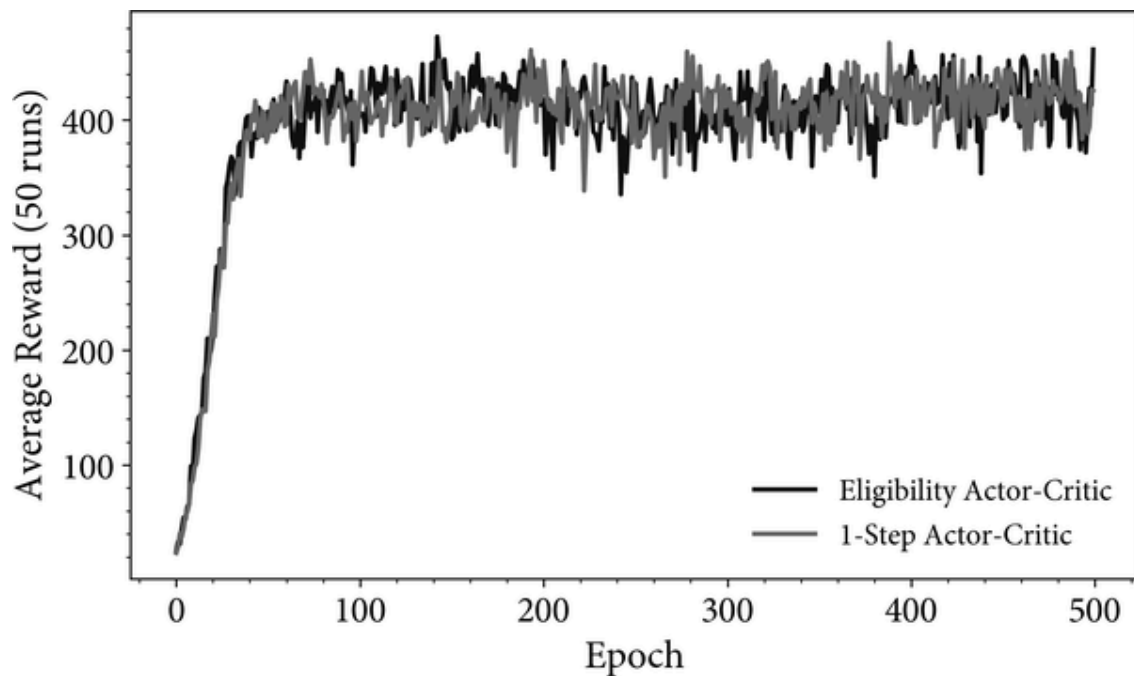


Figure 5-8. The average rewards for the `CartPole` environment for the 1-step actor-critic and eligibility traces algorithm.

The performance is largely unchanged. This demonstrates that a successful agent does not require foresight in the `CartPole` environment. Observing the potential downfall at the next step is adequate.

Changing the value of λ has little effect except for values of 0, where there is no temporal-difference update, and 1, where gradients explode. In my experiments, all values in between led to similar results.

In environments where rewards are infrequent or a significant amount of planning is required, multistep temporal-difference estimates excel.

A Comparison of Basic Policy Gradient Algorithms

This chapter has presented four different implementations of the policy gradient algorithm. All use gradient descent to iteratively improve the policy, but each have different value estimate implementations. A better value estimate reduces unnecessarily large updates; it reduces the variance of the gradients.

You can see the different forms in [Equation 5-11](#). The gradient of the policy is equal to the expectation of the gradient of the parameters multiplied by the return. The only thing that differs between these algorithms is how you quantify the return. I find this high-level interpretation of policy gradient algorithms insightful because it clarifies the subtle changes between distinct implementations.

Equation 5-11. Equivalent implementations of the policy gradient algorithm

$$\begin{aligned}\nabla_{\theta} J(\theta) &\doteq \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) G] && \text{(REINFORCE)} \\ &\doteq \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) (G - V_w(s)) && \text{(REINFORCE with Baseline)} \\ &\doteq \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) Q_w(s, a)] && \text{(Actor-Critic)} \\ &\doteq \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) A_w(s, a)] && \text{(Advantage Actor-Critic)} \\ &\doteq \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) \delta] && \text{(Temporal-Difference Actor-Critic)} \\ &\doteq \mathbb{E}_{\pi_{\theta}} \nabla_{\theta} \ln \pi_{\theta}(a | s) \delta(\lambda)] && \text{(Eligibility Actor-Critic)}\end{aligned}$$

Industrial Example: Automatically Purchasing Products for Customers

A recent business model trend is to automatically deliver products to customers and allow them to return those that they are not interested in. Stitch Fix is one of the most famous of the *subscription box model* businesses. After submitting a smorgasbord of initial information about body type and personal preferences, a Stitch Fix customer will be sent a package full of new clothes. The customer is then free to send back any clothes they don't like and they are charged for the items they keep (plus a service fee).

This idea might sound novel, but companies have been doing this for a long time via “no-commitment” subscription models. For example, Hewlett Packard will happily deliver ink for your printer at a prescribed rate. You can subscribe to certain products on Amazon. Hello Fresh will deliver you food. The difference between no-commitment and subscription box models is that if the customer is on the wrong plan (for example, they are receiving too much ink), they are likely to cancel. It is in HP's interest to deliver ink at the right rate to maximize the customer lifetime value.

I want to stress how impressive this challenge is. Here, an agent with no human intervention can automatically send you products and iteratively learn your preferences. You will receive items when you need them without you ordering them. Even better, the agent knows what items you might like, even though you have never thought of buying them yourself. From the perspective of a marketplace, this demonstrates why RL could be so lucrative. Vendors can sell you more stuff and you will be genuinely happy that they have!

The Environment: Gym-Shopping-Cart

This example embodies the subscription box model using publicly available data. I have created [an environment](#) that uses data from Instacart to simulate purchasing behavior.

Instacart [open sourced three million orders](#) from its shopping platform. I performed a significant amount of data cleaning that resulted in a state consisting of one-hot encoded day of the week and hour of the day. It also includes the number of days since the last order normalized to a value of 1. The actions are a list of 50,000 one-hot, binary encoded products. To order a product you set the product's column to 1. I keep the reward simple. You will receive a reward of +1 for every product that matches what the customer actually ordered. You will receive a reward -1 for every incorrectly ordered product. The goal is to order what the customer desires and nothing else. Each order is a step and all orders for a single customer is an epoch.

This is a simplification over what happens in the real world. The actual rewards are unlikely to be as simple as ± 1 . Offering a new product to a customer might cause them to buy more in the future, whereas a mistake could cost a lot if the products are perishable and can't be resold. I am limited by the features offered in the dataset. Other features are likely to be more informative. For example, Stitch Fix has a library of your preferences. You should build a process to gather user preferences about what food they like.

But the greatest issue is that the simulation uses static data. Ideally the agent should explore which products the customer desires. The customer might not even know it yet. But with fixed data, you are optimizing to solve a static solution. In other words, you can use standard machine learning techniques. Actionable exploration is at the heart of RL and in this example there is none. RL is more than just prediction.

One potential solution to this is to model customer purchases. For example, you can build a generative adversarial network to model what customers purchase in a simulated, but dynamic environment. Then you could transfer the learned policy to a live environment (see [“Further Reading”](#)). For the sake of simplicity, I use the static data.

Expectations

The state and actions in this environment are binary, so you could use a tabular technique like Q-learning. But to continue the thread of this chapter I will use the eligibility actor-critic policy gradient algorithm (see [“Eligibility Traces Actor-Critic”](#)). The agent must generate the same number of binary actions as products. I use a linear policy with a logistic function for every action. Note that softmax is not appropriate because that normalizes all actions to have a total value of 1. I also model the value function as a linear function. Both are for the sake of simplicity. You could use more complex models.

Let me stop for a moment to set some expectations. The propensity to buy a product is a function of the customer’s profile. You can imagine that if the customer had babies then they might be more likely to buy baby products. If they are vegan then they are unlikely to order meat. If they are young they will probably buy different products compared to someone who is old. All of these are latent features. They are not directly observable from the actions that they take. It is for this reason that it is unlikely that the linear model will perform very well in a real-life test. The linear model will learn how often customers buy products, ignoring *why* they buy them.

TIP

Simplify the problem by reducing the number of latent features. Create explicit features.

In a real application then, it will pay dividends if you invest in building a more sophisticated model that represents the customers, not just the products. Given enough data, you should also look at deep learning to represent these higher-level features.

Another issue is that the linear model has no memory. When I bulk buy washing powder, I rarely buy more in the next shop. The exception is when the kids decide to build a bed-sheet den in the mud. But the linear weights see that I bought lots of washing powder and probably offer more. To learn that people buy products at different rates you have to introduce some memory. One of the simplest ways to achieve this is to in-

clude the previous orders in the state. This is how DQN works so well on the Atari environment, for example. The researchers found that four frames worth of pixels were enough to provide a proxy for memory. More advanced solutions include using stateful neural networks like recurrent and long short-term memory neurons (see [“Common Neural Network Architectures”](#)).

Results from the Shopping Cart Environment

Again, for the sake of simplicity, I stick with a linear implementation. To help with the scale of the problem—50,000 actions—I allow the agent to buy only the most popular products.

You can see the first results in [Figure 5-9](#). In this initial experiment I restricted the dataset to a single customer. I chose to keep the 15 most popular products for this customer and trained the agent over 50 epochs. You can also see an agent that randomly orders products, for reference. In this example the customer had 358 purchases of popular products over 71 orders. Hence the highest reward possible is 358.

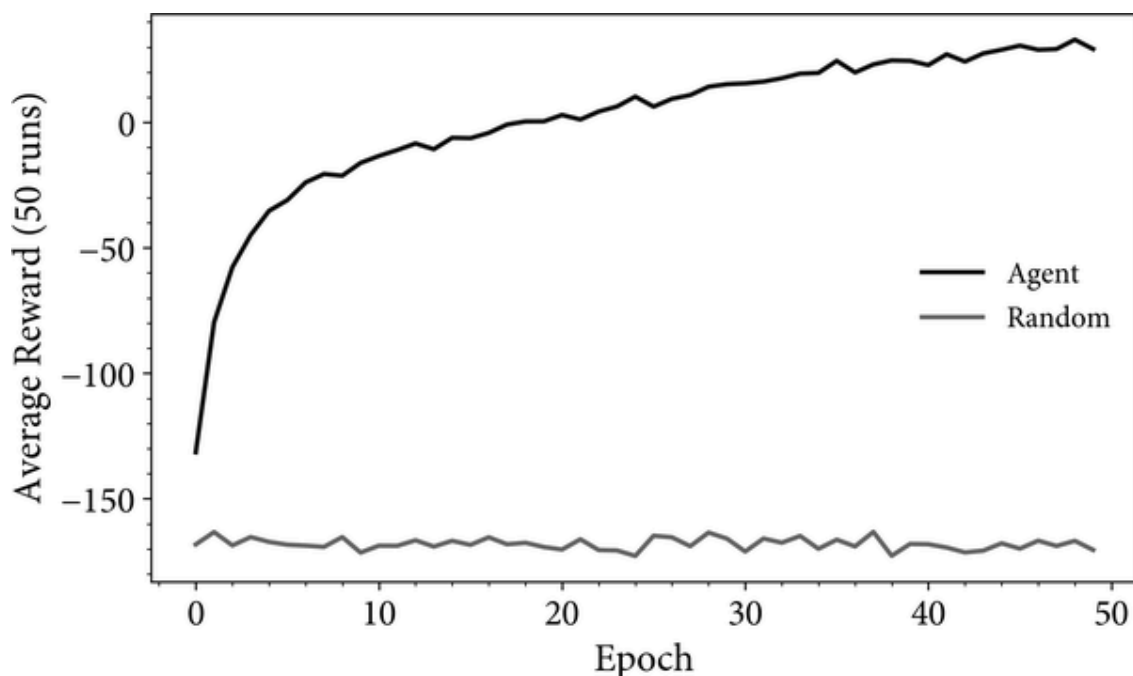


Figure 5-9. The average rewards for the shopping cart environment for a single customer (repeated 50 times) and for the 15 most popular products.

The highest reward for the agent was 57, indicating that the agent was buying products that the customer actually wanted. But this is much lower than the highest possible reward of 358 products. The random agent performs consistently poorly because it repeatedly orders products the customer does not want.

In [Figure 5-10](#) I repeat the same experiment but this time with 50 random customers that have different numbers of orders. In other words, the agent cannot overfit a single customer. It must attempt to generalize to all customers. This time the agent achieves a maximum reward of -1 . A little underwhelming compared to the previous experiment. This is saying that the agent is capable of just about breaking even for any given random customer. But you must remember how simple the agent is. The agent has only a handful of linear parameters and it is, to a limited extent, able to capture the complexity of the shopping process. I find this remarkable.

[Figure 5-11](#) shows the performance of the same agent over random customers when altering the number of products that the agent can choose from. In other words, I am altering the number of actions. When the number of products increases the agent receives less reward. However, it does appear that performance is asymptotic. Performance may converge with more training time.

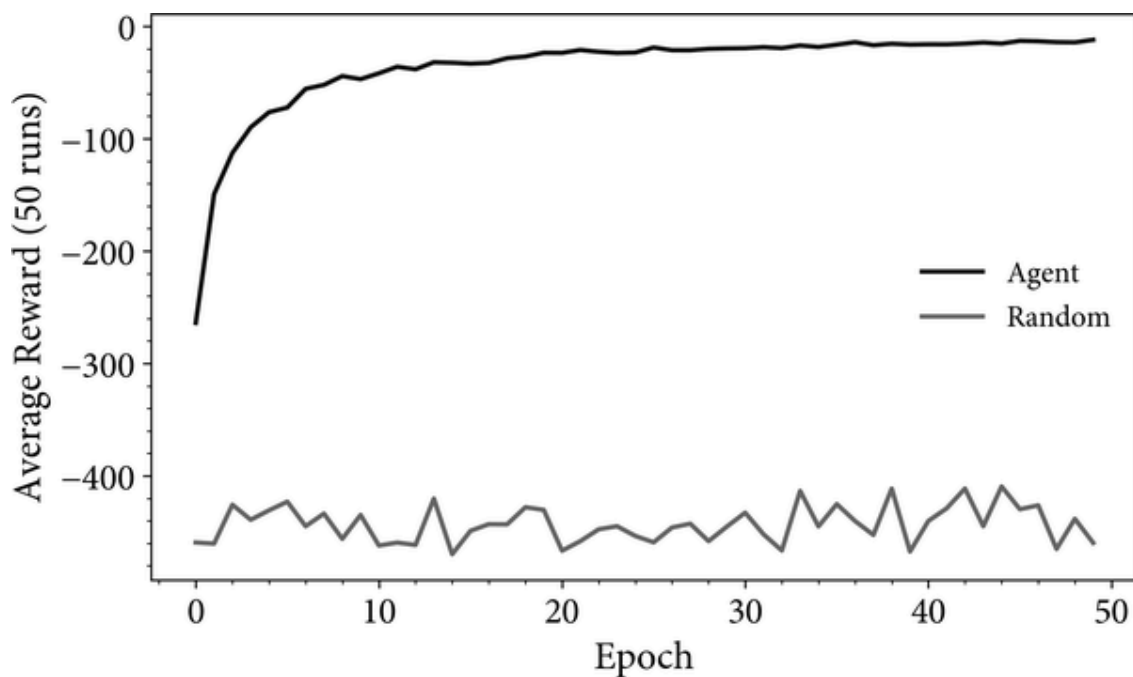


Figure 5-10. The average rewards for the shopping cart environment for 50 random customers (50 epochs) and for the 15 most popular products. The highest reward was -1 .

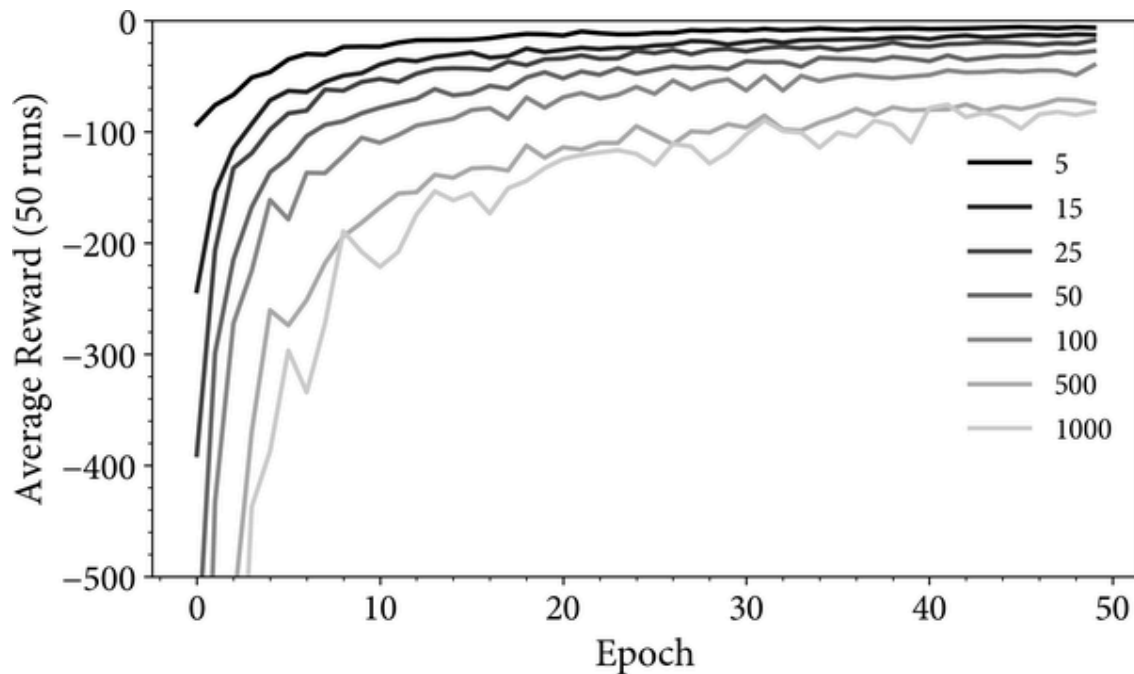


Figure 5-11. The average rewards for the shopping cart environment for 50 random customers (50 epochs) while altering the number of most popular products.

Finally, I want to demonstrate something called *transfer learning*. The goal is to train a model to solve one problem then apply the same model to a different, but related problem. This paradigm works really well with RL because you can potentially learn from offline data or models and fine-tune using online data.

For example, I can use the shopping cart model trained upon random customers (Figure 5-10) then transfer the learned model to a new customer.

Figure 5-12 shows the results of such an experiment.

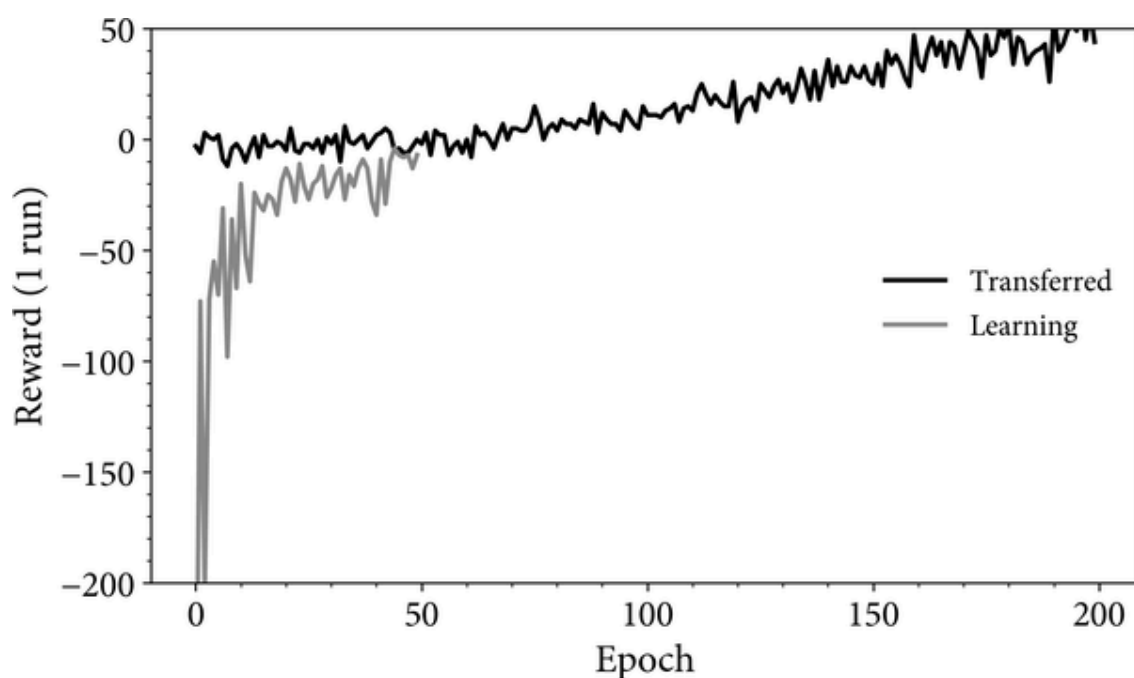


Figure 5-12. The average rewards for the shopping cart environment for a single customer after initially training upon random customer data and transferring the model (15 products, 1 run).

To transfer the model I copied the policy and value estimate weights learned when training on random customers and pasted them into another agent with only a single customer. Also, these results are for a single run. You can imagine a smoother set of results if I were to repeat this 50 times. If you compare this result to [Figure 5-9](#), then you can see there is no initial ramp-up in performance because the pretrained weights are already very good.

Another observation is that it takes a long time (approximately 200 epochs) for the single customer agent to obtain similar performance to the result in [Figure 5-9](#). This is because the agent has learned a policy that works best on average across all customers. When I transfer the policy across to a new agent on the single customer data, it needs to retrain those weights. But the average policy appears to have lowered the probability of exploring the states that are most important to that single customer.

In other words, it seems that in [Figure 5-9](#), the agent has overfit for that particular customer. Typically overfitting is a bad thing, since the model will not generalize to other customers. But in this example, that is exactly the point. I want the agent to fit around the preferences of a single customer.

Here there is a trade-off. You want to start with a reasonable policy, so the customer receives good recommendations quickly. But you don't want to have to wait for 200 orders before they are in positive reward territory. One solution to this problem is to treat the number of pretraining epochs as a hyperparameter and tune it to obtain a satisfactory result.

Summary

Prior to this chapter, I focused on value-based methods, methods that quantify the value of being in a state. The policy is derived as a maximization of the expected value in each state. But another way is to parameterize the policy directly and alter the weights of the parameterization to produce a policy that maximizes the expected return. You can use policy gradient methods to find the updates necessary to optimize a policy.

Policy gradient methods have a range of advantages. They can learn to output probabilities, which negates the need for an explicit exploration algorithm like ϵ -greedy. They can output continuous actions. And sometimes it is easier to model the policy, rather than having to model the state-action value.

On the other hand, policy gradient methods tend to be more sensitive to the choice of model. With value methods, it is fairly easy to alter the complexity of the model to predict the expected value function, whereas the policy model needs to be chosen carefully.

This chapter followed the same derivation as before, from Monte Carlo through to eligibility traces. Even though policy-based methods are optimizing for a slightly different solution, expected value estimates are still incredibly important, as you saw in the actor-critic algorithms.

I presented a simplified industrial example where the goal was to automatically order products for users. You can imagine a large number of improvements to this and the generalization of this idea, recommenders, is an active area of RL research. I know of several organizations that are using RL for recommendation purposes.

Further Reading

- Gradient descent:
 - Ruder, Sebastian. 2017. [“An Overview of Gradient Descent Optimization Algorithms.”](#) ArXiv:1609.04747, June.
- Policy gradient derivation:
 - A succinct mathematical introduction to on-policy gradients: Kämmerer, Mattis Manfred. 2019. [“On Policy Gradients.”](#) ArXiv:1911.04817, November.
 - Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- Building generative models:
 - Foster, David. 2019. *Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play*. O’Reilly.

- 1** Houthoofd, Rein, Richard Y. Chen, Phillip Isola, Bradley C. Stadie, Filip Wolski, Jonathan Ho, and Pieter Abbeel. 2018. [“Evolved Policy Gradients”](#). ArXiv:1802.04821, April.
- 2** Bhatnagar, Shalabh, Mohammad Ghavamzadeh, Mark Lee, and Richard S Sutton. 2008. [“Incremental Natural Actor-Critic Algorithms”](#). In *Advances in Neural Information Processing Systems 20*, edited by J. C. Platt, D. Koller, Y. Singer, and S. T. Roweis, 105–112. Curran Associates, Inc.
- 3** Wolpert, D. H., and W. G. Macready. 1997. [“No Free Lunch Theorems for Optimization”](#). *IEEE Transactions on Evolutionary Computation* 1(1): 67–82.

Table of contents collapsed